2)

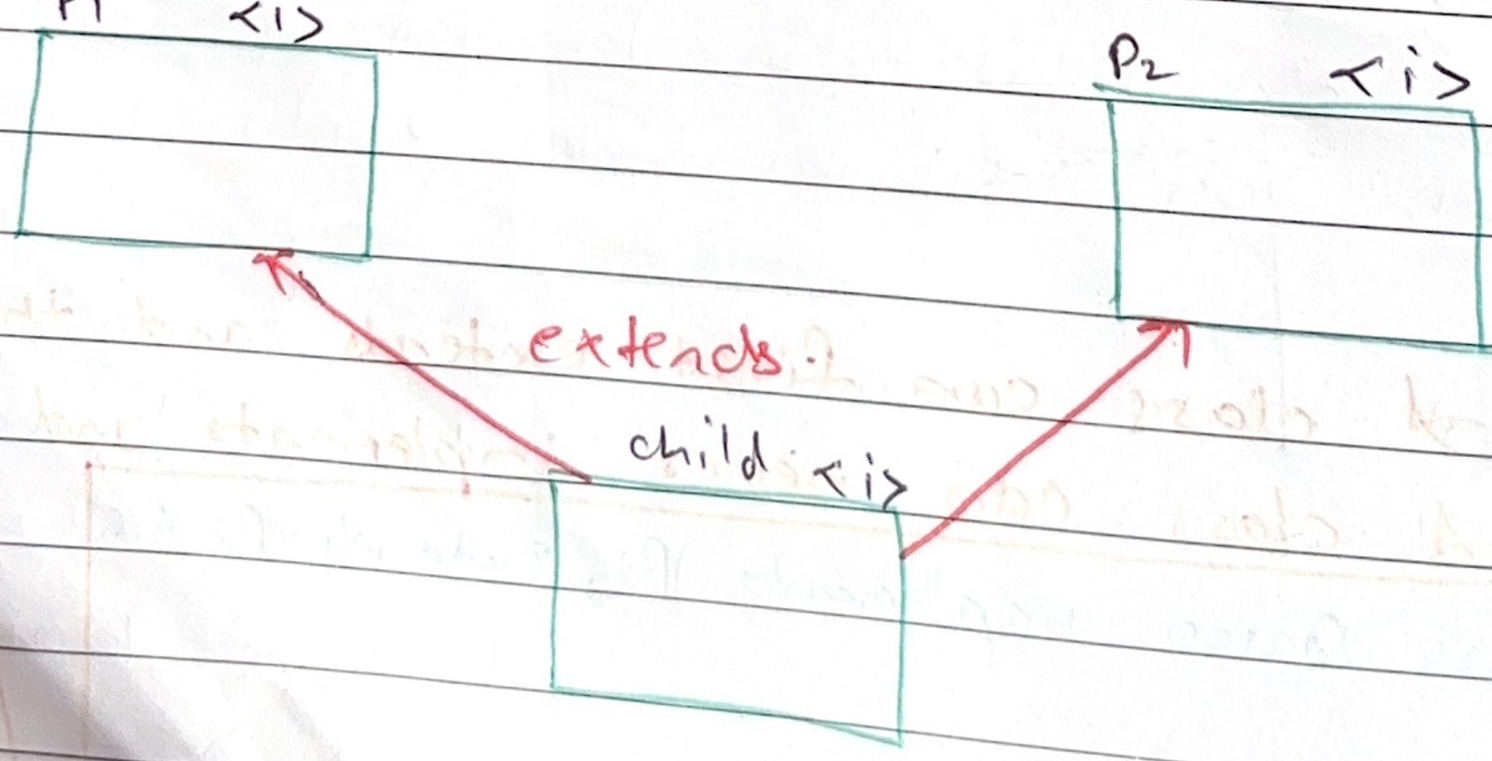



Eg :-

```
Package multipleInheritance.two;
public interface Parent1{

}
public interface Parent2{

}
public class Child implements Parent1, Parent2 {

}
```

Note :- A class can implements 'n' number of interfa at same levele..

3)



Eg:-

```
Package MultipleInheritance.three;
public interface Parent1 {

}
public interface Parent2 {

}
public interface Imp extends Parent1, Parent2 {

}
```

Note:- An interface can extends 'n' number of interfaces at same level.

* What members are inherited from interface to class?
  1) Variables
  2) Abstract methods
  3) Default methods

* What members are not inherited from interface to class?
  1) Static methods.
  2) Private methods → non static method
  3) Private static methods

* Why static methods are not inherited from interface to class?

→ To avoid ambiguity and conflicts.
  If two interface have static methods will the same

name, and a class implements both, it would create confusion about which one to use

To prevent this ambiguity, Java does not allow static methods in interface to be inherited. ~~They must~~

They must always be called using the interface name. (Eg. InterfaceName.MethodName()).

```
interface Parent1{                    interface Parent2{
    public static void m1(){              public static void m2(){
    }                                     }
}                                     }

class child implements Parent1, Parent2 {
    public static void main (String[] args){
        child cl = new child();
        cl.m1();
    }
}
```

CTE because of ambiguity ~~ess~~ issue

* What is the purpose of ~~abstract~~ constructor inside the abstract class when we are not able to create the object..?

→ Even know we are not able to create the object, we can create object of its child class, to load ~~members~~ ~~of~~ non-static members of abstract class, into child class object we need constructor inside the abstract class.

As we
the parent
object clas
a parent t
should cal
why we
class.

* Difference

Abstr

1) Can have
and concrete

2) Can have n
variables

3) A class can
abstract c

4) Can can ho

5) Methods ca
access modif

As we already ~~at~~ know for every object class is the parent class, similarly for abstract ~~at~~ class also object class is parent class. Since object class is a parent the child which is abstract must and should call the constructor of object class. thats why we need a constructor inside ~~obj~~ abstract class.

* Difference between abstract class and interface

| Abstract class | Interface |
|---|---|
| 1) Can have both abstract and concrete methods | ~~Unty~~ Can have abstract, default, static and private methods |
| 2) Can have non-static ~~var~~ variables | All variables are by default public static final. |
| 3) A class can extends only one abstract class | A class can implements multiple interface. |
| 4) Can can have a constructor | Cannot have a constructor. |
| 5) Methods can have any access modifier | All abstracts methods ~~or~~ are public by default. |

10/11/2025

* **String:-**

**1) String literal:-**

In java string literal is a group of characters enclosed within double quote (" ");

String literals in java are ~~or~~ stored using object of string class.

There are three built in classes to store string literals

1) java.lang.StringBuilder
2) java.lang.StringBuffer
3) java.lang.String

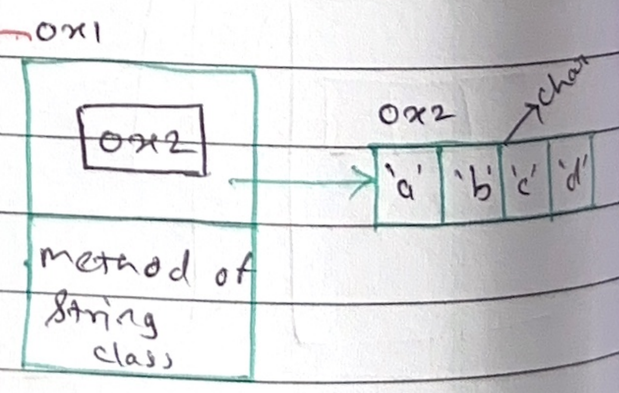
**1) java.lang.String :-**

- String is a built in class which is present in java.lang package
- String class is immediate child class of object class.
- String class is declared as final which means it cannot be inherited.
- toString() of object class is overriden in String class which returns the string value present in ~~str~~ given String object.

Eg:-

```
String s = "abcd"
```

(0x1)




```
Sopln(s);// abcd
```

internally calls to String method.

Eg. code

```
class Demo
{
    String s =
    public stat
    {
        String
        String
        Sopln(
        Sopln(
        Sopln(s
    }
}
```

Whenever one ~~what~~ obje
String const
same strin
be return.
If the
scp the

Note:- String
heap area
Insi
object will

Eg. Code

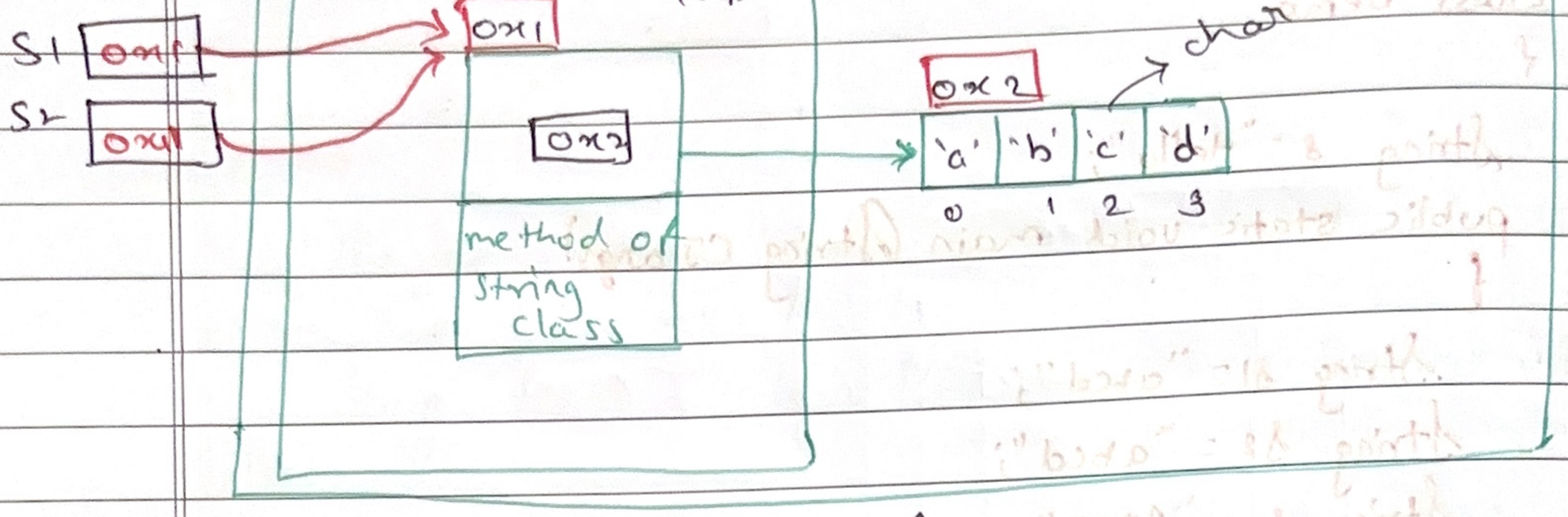
```
class Demo
{
  String s = "Hello";
  public static void main (String [] args)
  {
    String s1 = "abcd";
    String s2 = "abcd";
    String s3 = "ABCD";
    System.out.println (s1);
    System.out.println (s2);
    System.out.println (s1==s2); // compares address.
  }
}
```

Whenever jvm encounter a String literals internally one object of String class will be created inside String constant Pool area [SCP]. If and only if their is no same string literal inside SCP and the reference will be return.

If the same literal already exist inside the SCP the reference of old literal will be returned.

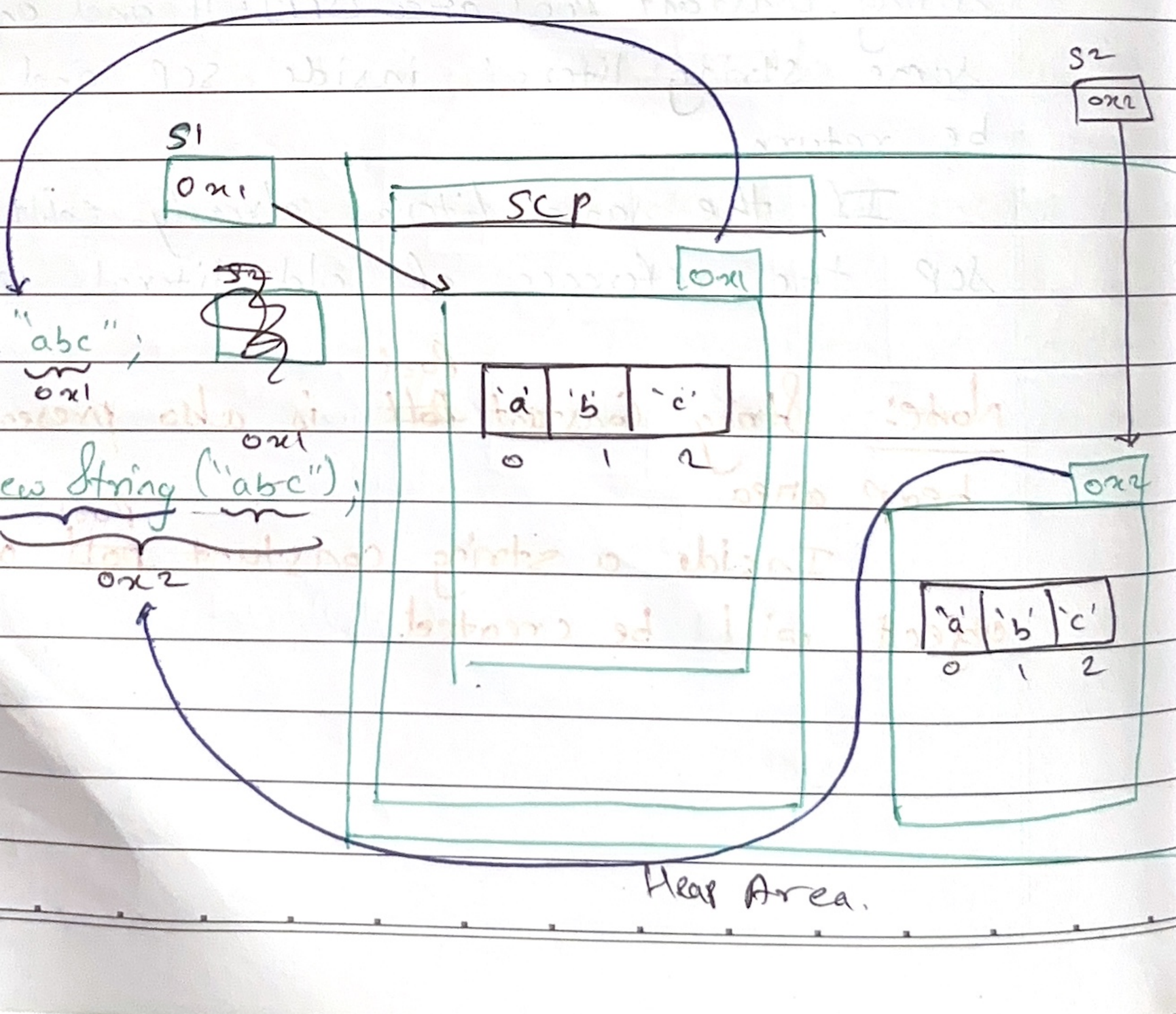
Note:- String Constant Pool is also present inside heap area.

Inside a string constant Pool only string class object will be created.



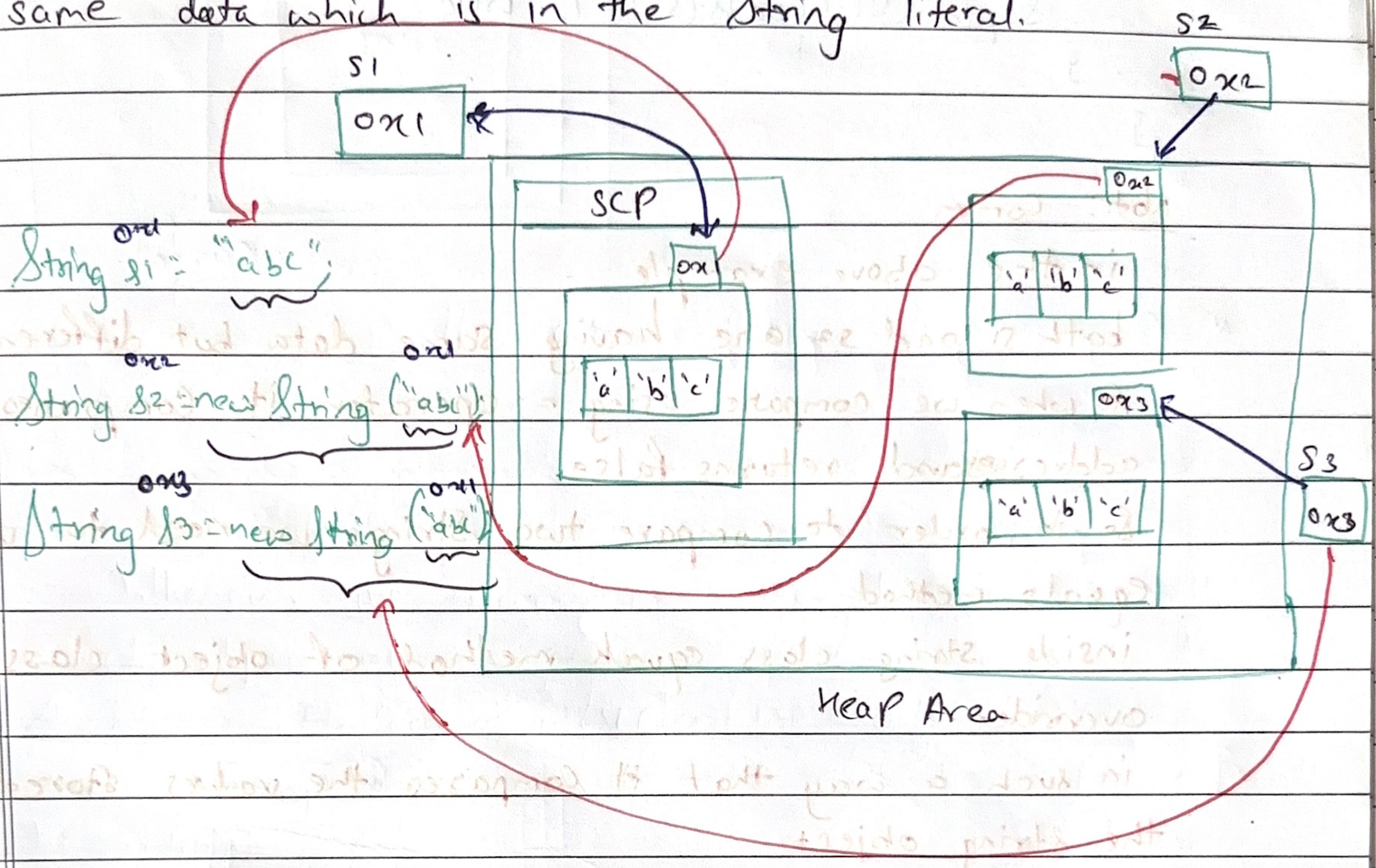

## * Important constructors of String class :-

1) String ()
2) String (String s)
3) String (char[] arr)
4) String (StringBuffer sb)
5) String (StringBuilder sb).



11/11/2025

When we create a String object by using new operator and by passing String literal to String class constructor internally one object will be created for the literal "hello" inside String constant pool area and one more object will be created by new operator outside the String constant pool and inside the heap area with the same data which is in the String literal.




Note:-

Inside String constant pool duplicate String objects are not allowed

Outside String constant pool, ie. inside heap area duplicate string objects are allowed.

```
class Demo {
  public static void main (String [] args)
  {
    String s1 = "abc";
    String s2 = new String ("abc");
    Sopln (s1 == s2); // false
    Sopln (s1.equals (s2)); // true.
  }
}
```

Note:- Comm

in the above example
both s1 and s2 are having same data but different address
So when we compare using == operator it Compares.
addresses and returns false
So in order to compare two Strings we need to use equals method
inside string class equals method of object class is overriden.
in such a way that it compares the values stored inside the string object.

Eg:-

```
class Demo {
  public static void main (String [] args)
  {
    String s1 = "ab";
    Sopln (s1.hashCode ());

    String s2 = "abc";
    Sopln (s2.hashCode ());
  }
}
```

O/P:-

97

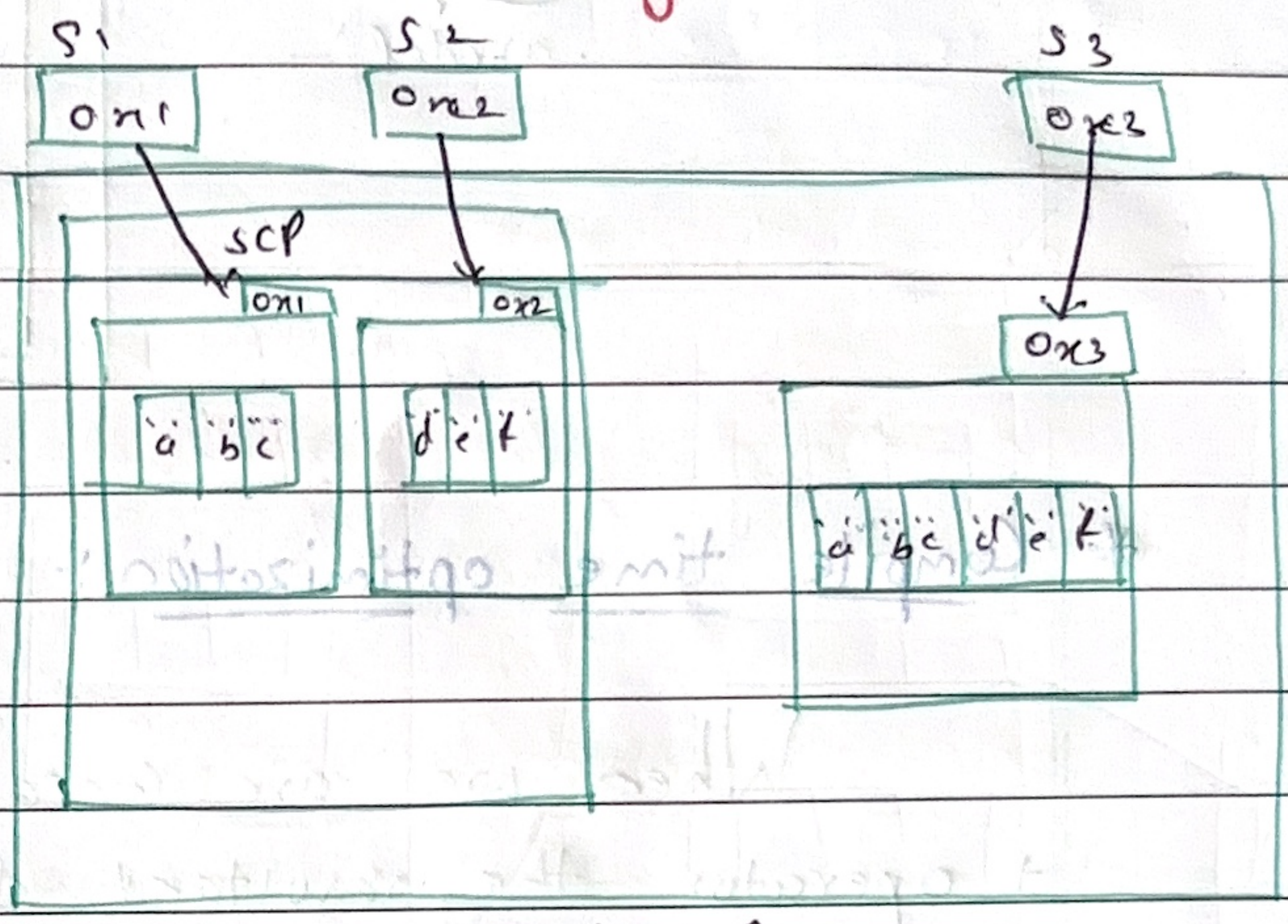
96374

Note:-

hashCode() of object is overriden in String class which returns an integer value that is generated based on String literal.

1)

```
String s1 = "abc";
String s2 = "def";
String s3 = s1 + s2;
```

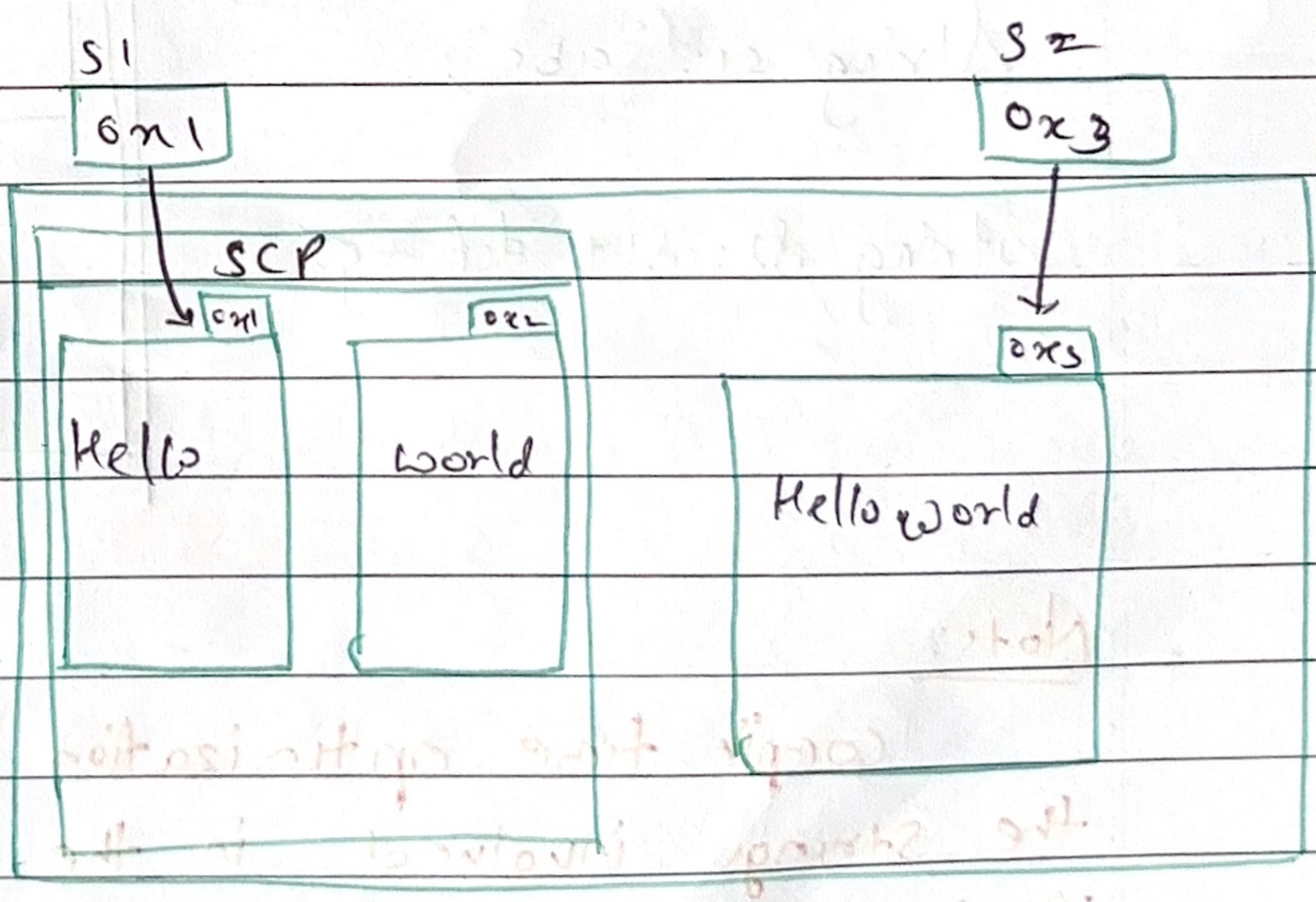



Heap Area

When we are concatnating 2 reference variables using plus (+) operator then the resultant string will be created inside heap area.

2)

```
String s1 = "Hello";
String s2 = s1 + "world";
```



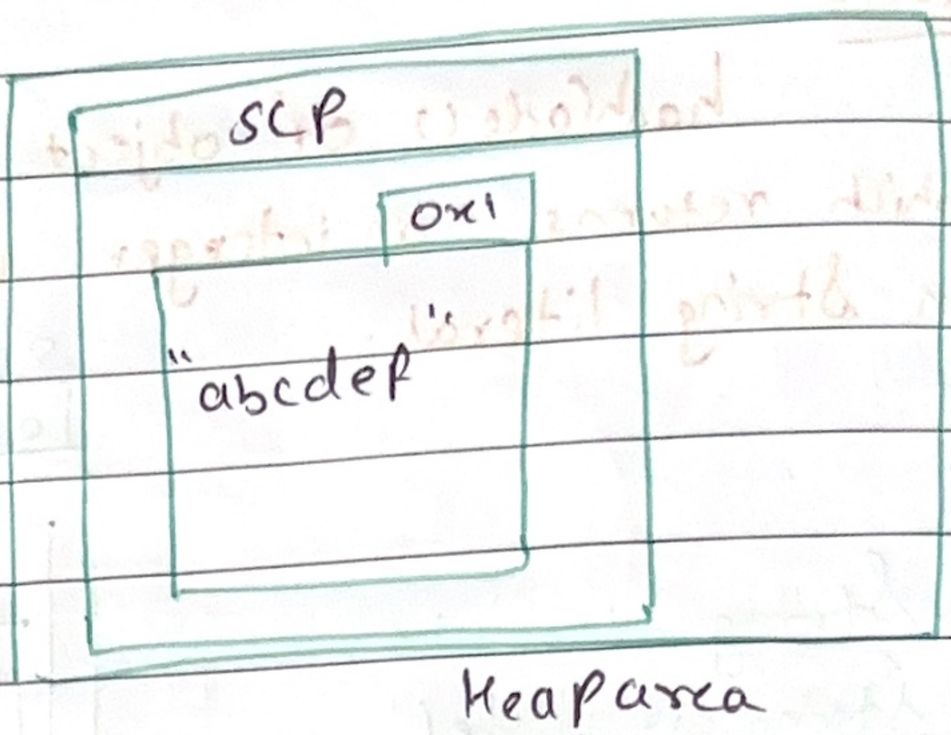

Heap area

When we are ~~cont~~ concatenating one string literal and the reference variable then the resultant String will be created inside heap area.

3]

```
String s1 = "abc" + "def";
```

"abcdef" — 0x1

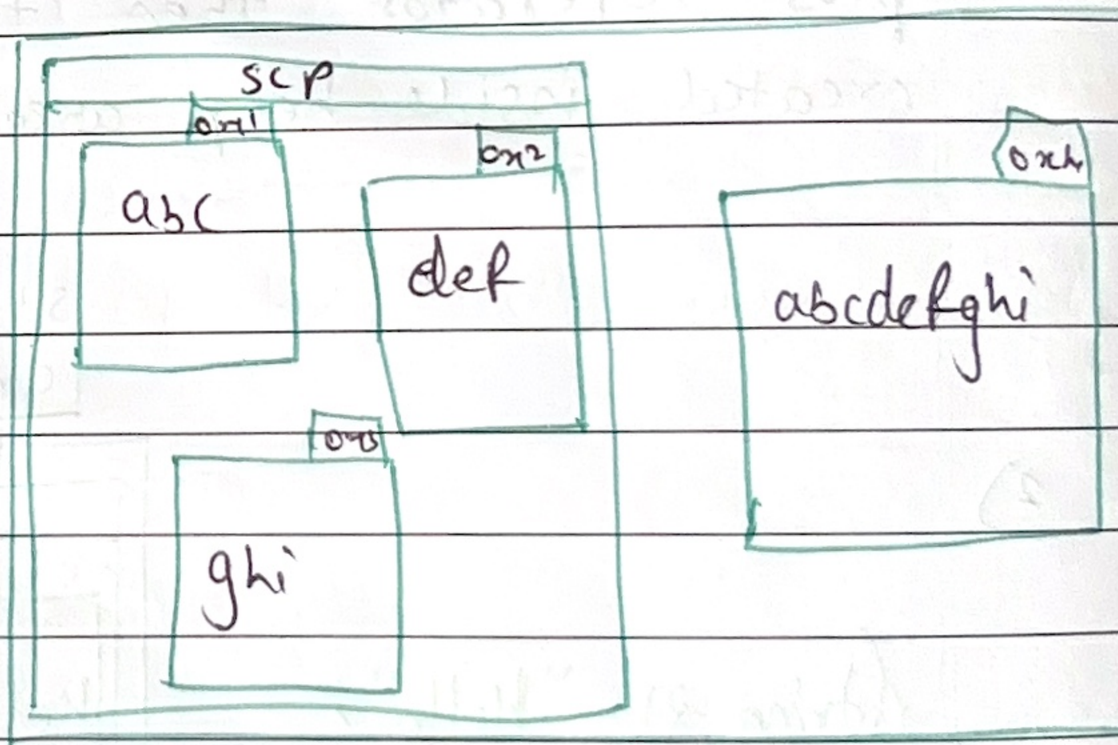



## * Compile time optimization :-

When we are concatenating 2 string literals using + operator the resultant String literal will be created inside string constant pool. only.

4]

```
String s1 = "abc";
String s2 = s1 + "def" + "ghi";
```

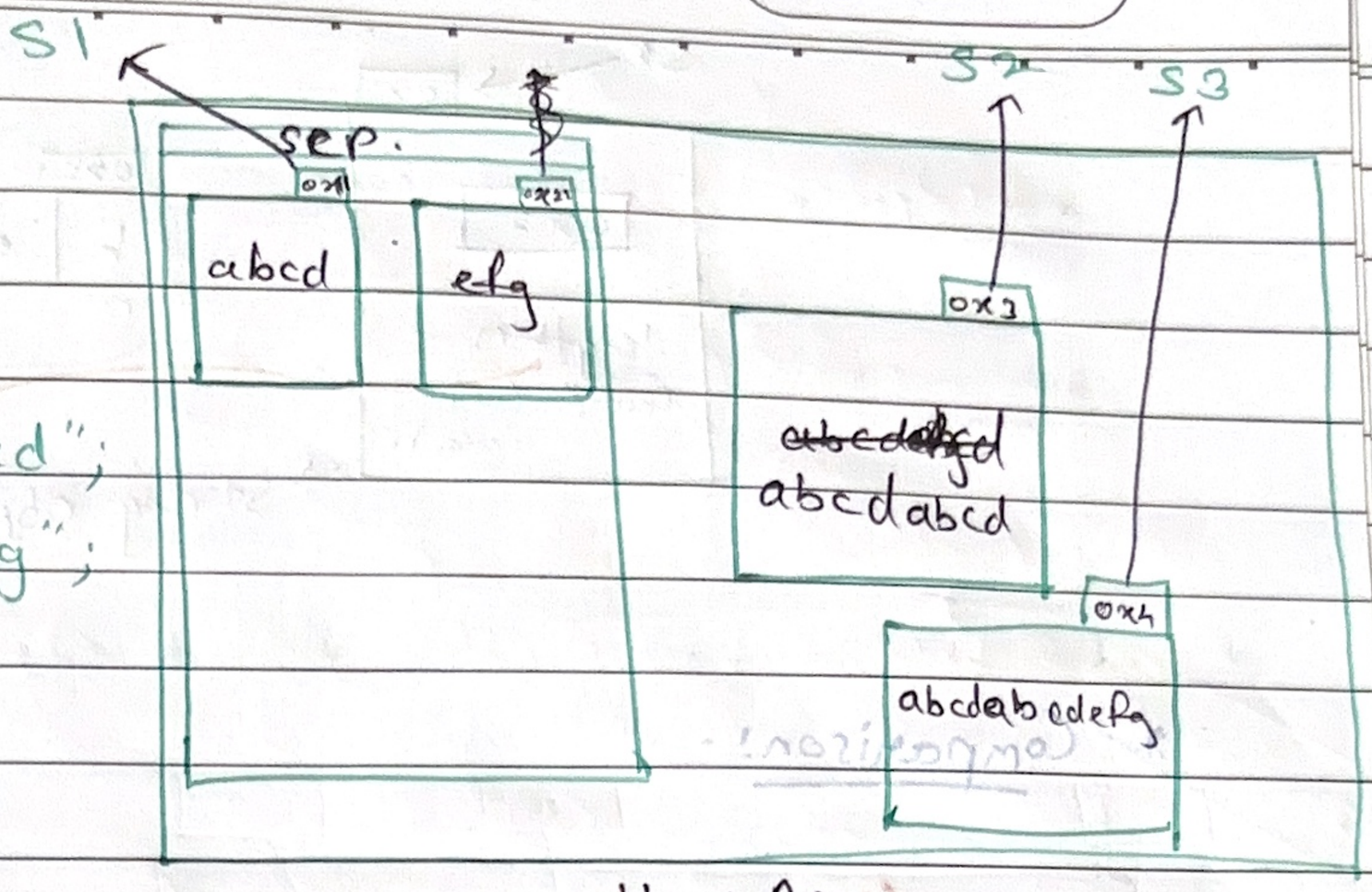



Note:-

compile time optimization happens if and only if all the strings involved in the concatenation are literals if there is reference variable involved in concatenation compile time optimization will not occur.

12/11/2025

```
String s1 = "abcd";
String s2 = s1 + "abcd";
String s3 = s2 + "efg";
```




* ~~Basic Information and Access~~:-

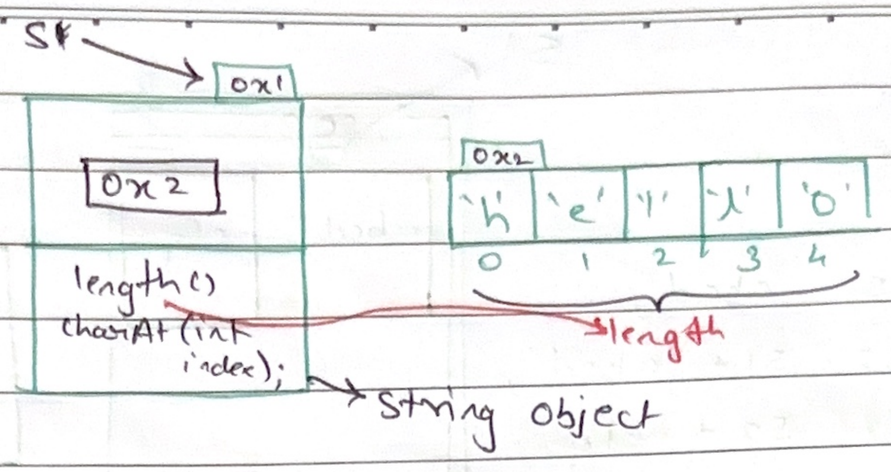
* Methods of a String class :-

- length() :- Returns the number of characters in the string.

- charAt (int index) :- Returns the characters at the specified index (0-based) (crucial for accessing individual character.

Eg :-

```
class Demo{
    public static void main (String [] args) {
        String s = "hello";
        Sopln (s.length()); //5
        Sopln (s.charAt(2)); //l
        Sopln (s.charAt(5)); //StringIndexOutOfBoundException
    }
}
```




* Comparison:-

- equals (String str):-

Compares the string to another string for exact equality (case-sensitive).

Use equals (String str) for string comparison, not ==;

- equalsIgnoreCase (String str):-

Compares current string to passed string, ignoring case.

* Searching and Finding:-

- indexOf (char char):- Ret

Returns the index of the first occurrence of a character.

- indexOf (String str):-

Returns the index of the first occurrence of a substring.

- indexOf (String str, int fromIndex):-

Returns the index of the first occurrence of a substring, starting the search at the specified index.

- lastIndex (String str)

Returns

a substring.

- StartsWith (String

Checks

Specified prefix.

- StartsWith (String

checks

prefix, beginning

- endsWith (String

Checks

suffix.

* Modification and

- concat (String st

Concate

generally prefer

- substring (int beg

Extract

- replace (char o

String newStri

- toLowerCase()

- lastIndex(String str) :-
  Returns the index of the last occurrence of a substring.

- StartsWith (String Prefix):-
  checks if ~~String~~ the string starts with the specified prefix.

- StartsWith (String Prefix, int ~~o~~offeset) :-
  checks if the string starts with the specified prefix, beginning at the specified index.

- endsWith (String ~~Prefix)~~ suffix):-
  Checks if the string ends with the specified suffix.

* Modification and Transformation :-

- concat (String str) or + operator :-
  Concatenates (joins) two strings. The + operator is generally preferred.

- substring (int beginIndex) or substring (int beginIndex, int endIndex) :-
  Extract a portion of the string.

- replace (char oldChar, char newChar) or replace (String oldString, String newString) :- Replace characters or substrings.

- toLowerCase() :- Converts the string to lowercase.

- toUpperCase():- Converts the string to uppercase.
- trim():- Renove leading and trailling whitespace.
- split(String Index):- Splits the String into an array of Substring based on a regular expression.

Date 13/11/2025

Eg:-2

```
class Demo2 {
    public static void main (String [] args)
    {
        String s1 = "hello how are you";
        String s2 = "Hello HOW ARE YOU";
        System.out.println (s1.equals(s2)); //false
        System.out.println (s1.equalsIgnoreCase(s2)); //true

        String s3 = "abcdef";
        System.out.println (s3.indexOf('b')); // 1
        System.out.println (s3.indexOf('h')); //-1
        System.out.println (s3.indexOf("def"); // 3
        System.out.println (s3.indexOf("defg"); //-1

        String s4 = "abcdefghijkl";
        System.out.println (s4.indexOf('d'): ("def", 1); //3
        System.out.println (s4.indexOf ("def", 4); //-1

        String s5 = "abcdefghidel";
        System.out.println (s5.lastIndexOf ('d')); //12
```

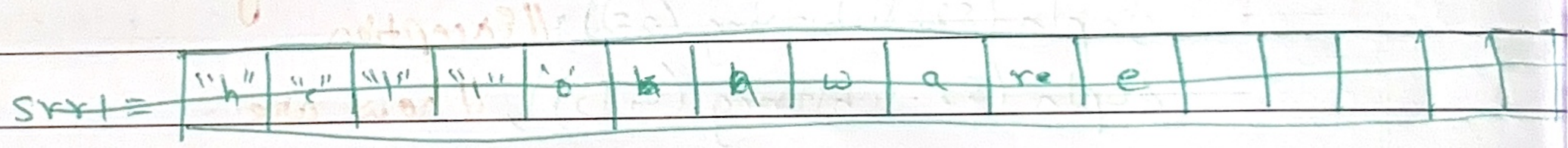
```
String s6 = "food mela on Saturday";
Sopln(s6.startsWith("food"); //true
Sopln(s6.startsWith("mela"); //false
Sopln(s6.endsWith("Saturday"); //true
Sopln(s6.endsWith("mela"); //false.
}
}
```

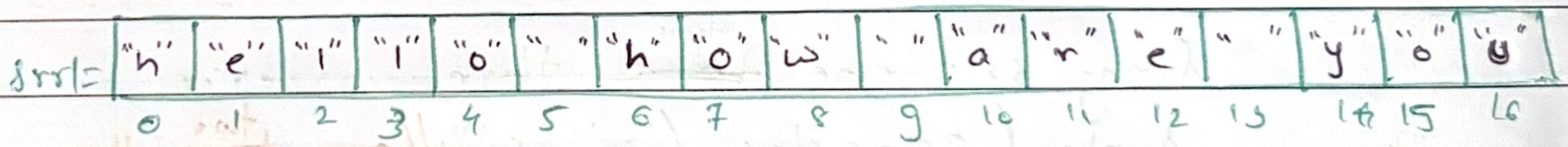
Eg 3]

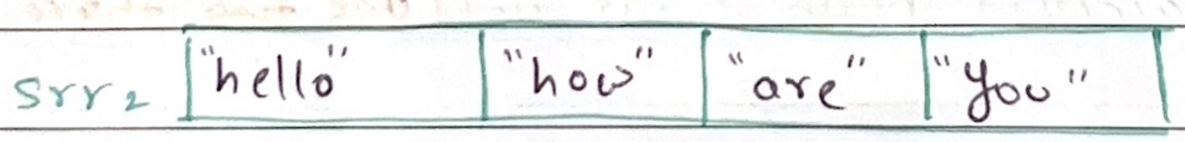
```
class Demo3f
{
    public static void main(String [] args)
    {
        String s1 = "abcdeF";
        Sopln(s1.concat("ghi")); //abcdeFghi

        Sopln String s2 = "hello how are you";
        Sopln(s2.substring(6)); //how are you
        Sopln(s2.substring(25)); //Exception
        Sopln(s2.substring(6,13)); //how are

        String s3 = "one two three rrrrrrrr";
        Sopln(s3.replace('r','e')); //one two theee eeeeeeee
        Sopln(s3.replaceAll("hello","rrr")); //one two three rrrrrrrr

        String s4 = "ABCdeF";
        Sopln(s4.toLowerCase()); //abcdef:

        String s5 = "ABCdef";
        Sopln(s5.toUpperCase()); //ABCDEF.

        String s6 = "  hello how are you  ";
        Sopln(s6.trim()); //hello how are you.
    }
}
```

Eg 4J.

```
class Demo4{
public static void main (String [] args)
{
    String s1 = "hello how are you";
    System.out.println (s1.length());

    String [] srr1 = s1.split(""); // passing empty string
    System.out.println(srr1.length);
    System.out.println (srr1[0]); //h
    System.out.println (srr[1]); //e

    String [] srr2 = s1.split(" "); // passing space
    System.out.println (srr2.length);
    System.out.println (srr2[0]); //hello
    System.out.println (srr2[1]); //how
}
}
```



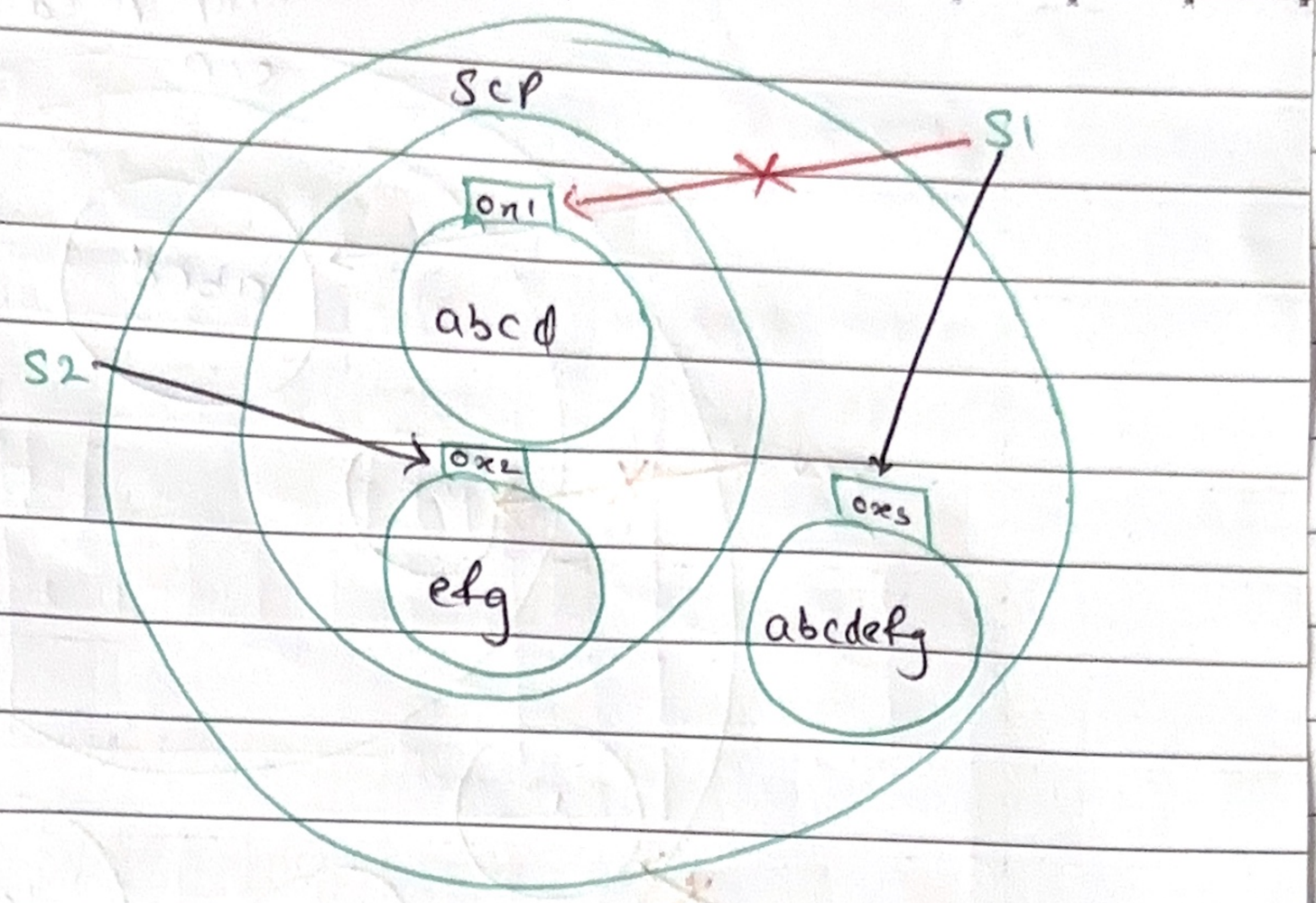




**Note:-**

In java String is immutable in nature once it is created it can never be modified.

```
String s1 = "abcd";
String s2 = "efg";
s1 = s1+s2;
sopln(s1);
```



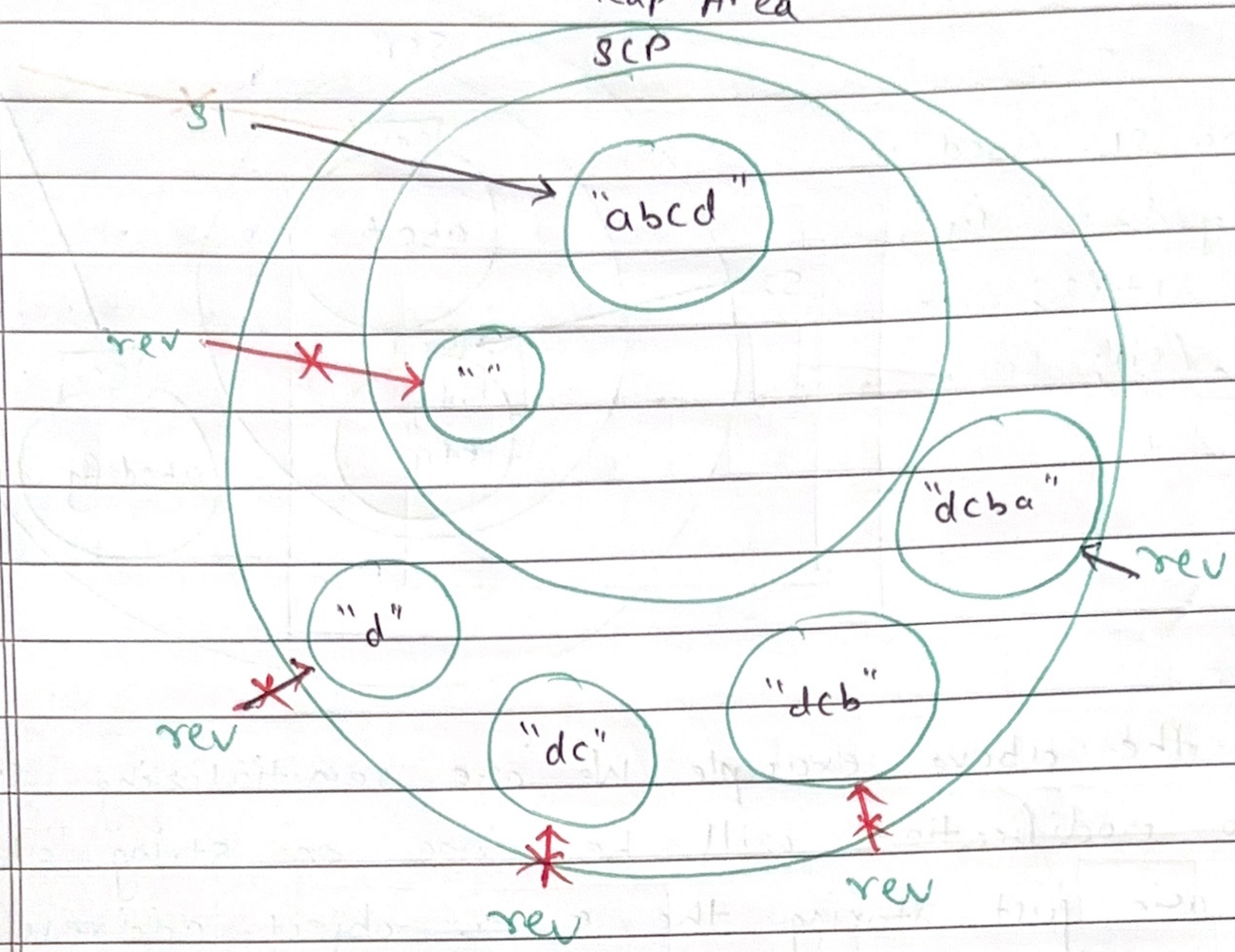

In the above example We are reinitializing s1 variable where no modification will be done on string "abcd" and we are just storing the new object address inside s1 variable.

Whenever we are dereferencing string variables every time new string objects will be created inside the heap area. which will occupy more memory scape ~~of~~ to overcome this we will make use of stringBuffer or stringBuilder.

Eg 5

```
class Demo5 {
    public static void main (String [] args)
    {
        String s1 = "abcd";
        String rev = "";
        for(int i = s1.length()-1; i>=0; i--) {
        {
            rev = rev + s1.charAt(i);
        }
    sopln (rev);
    }
}
```

"" + 'd' → "d" 1st
"d" + 'c' → "dc" 2nd
"dc" + 'b' → "dcb" 3rd
"dcb" + 'a' → "dcba". 4th


Heap Area
SCP
s1
"abcd"
rev
""
"dcba"
rev
"d"
rev
"dc"
"dcb"
rev
rev